11.5 (2,4) Trees

In this section, we will consider a data structure known as a ***(2,4) tree***. It is a particular example of a more general structure known as a ***multiway search tree***, in which internal nodes may have more than two children. Other forms of multiway search trees will be discussed in Section 15.3.

11.5.1 Multiway Search Trees

Recall that general trees are defined so that internal nodes may have many children. In this section, we discuss how general trees can be used as multiway search trees. Map entries stored in a search tree are pairs of the form (k,v), where k is the ***key*** and v is the ***value*** associated with the key.

Definition of a Multiway Search Tree

Let w be a node of an ordered tree. We say that w is a ***d-node*** if w has d children. We define a multiway search tree to be an ordered tree T that has the following properties, which are illustrated in Figure 11.22a:

- Each internal node of T has at least two children. That is, each internal node is a d-node such that $d \geq 2$.
- Each internal d-node w of T with children $c_1, \ldots, c_d$ stores an ordered set of $d-1$ key-value pairs $(k_1,v_1), \ldots, (k_{d-1},v_{d-1})$, where $k_1 \leq \cdots \leq k_{d-1}$.
- Let us conventionally define $k_0 = -\infty$ and $k_d = +\infty$. For each entry (k,v) stored at a node in the subtree of w rooted at c_i, $i = 1, \ldots, d$, we have that $k_{i-1} \leq k \leq k_i$.

That is, if we think of the set of keys stored at w as including the special fictitious keys $k_0 = -\infty$ and $k_d = +\infty$, then a key k stored in the subtree of T rooted at a child node c_i must be "in between" two keys stored at w. This simple viewpoint gives rise to the rule that a d-node stores $d-1$ regular keys, and it also forms the basis of the algorithm for searching in a multiway search tree.

By the above definition, the external nodes of a multiway search do not store any data and serve only as "placeholders." As with our convention for binary search trees (Section 11.1), these can be replaced by null references in practice. A binary search tree can be viewed as a special case of a multiway search tree, where each internal node stores one entry and has two children.

Whether internal nodes of a multiway tree have two children or many, however, there is an interesting relationship between the number of key-value pairs and the number of external nodes in a multiway search tree.

Proposition 11.6: *An n-entry multiway search tree has $n+1$ external nodes.*

We leave the justification of this proposition as an exercise (C-11.49).

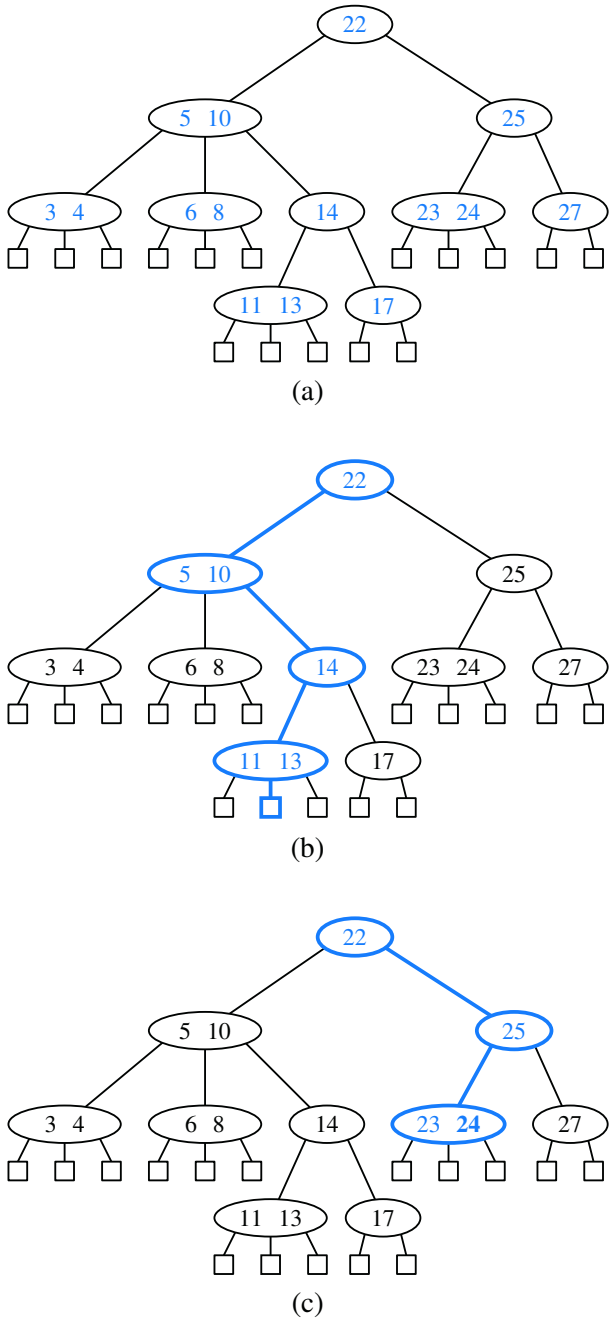

Figure 11.22: (a) A multiway search tree T; (b) search path in T for key 12 (unsuccessful search); (c) search path in T for key 24 (successful search).

Searching in a Multiway Tree

Searching for an entry with key k in a multiway search tree T is simple. We perform such a search by tracing a path in T starting at the root. (See Figure 11.22b and c.) When we are at a d-node w during this search, we compare the key k with the keys $k_1, \ldots, k_{d-1}$ stored at w. If $k = k_i$ for some i, the search is successfully completed. Otherwise, we continue the search in the child c_i of w such that $k_{i-1} < k < k_i$. (Recall that we conventionally define $k_0 = -\infty$ and $k_d = +\infty$.) If we reach an external node, then we know that there is no entry with key k in T, and the search terminates unsuccessfully.

Data Structures for Representing Multiway Search Trees

In Section 8.3.3, we discuss a linked data structure for representing a general tree. This representation can also be used for a multiway search tree. When using a general tree to implement a multiway search tree, we must store at each node one or more key-value pairs associated with that node. That is, we need to store with w a reference to some collection that stores the entries for w.

During a search for key k in a multiway search tree, the primary operation needed when navigating a node is finding the smallest key at that node that is greater than or equal to k. For this reason, it is natural to model the information at a node itself as a sorted map, allowing use of the ceilingEntry(k) method. We say such a map serves as a ***secondary*** data structure to support the ***primary*** data structure represented by the entire multiway search tree. This reasoning may at first seem like a circular argument, since we need a representation of a (secondary) ordered map to represent a (primary) ordered map. We can avoid any circular dependence, however, by using the ***bootstrapping*** technique, where we use a simple solution to a problem to create a new, more advanced solution.

In the context of a multiway search tree, a natural choice for the secondary structure at each node is the SortedTableMap of Section 10.3.1. Because we want to determine the associated value in case of a match for key k, and otherwise the corresponding child c_i such that $k_{i-1} < k < k_i$, we recommend having each key k_i in the secondary structure map to the pair (v_i, c_i). With such a realization of a multiway search tree T, processing a d-node w while searching for an entry of T with key k can be performed using a binary search operation in $O(\log d)$ time. Let $d_{\max}$ denote the maximum number of children of any node of T, and let h denote the height of T. The search time in a multiway search tree is therefore $O(h \log d_{\max})$. If $d_{\max}$ is a constant, the running time for performing a search is $O(h)$.

The primary efficiency goal for a multiway search tree is to keep the height as small as possible. We will next discuss a strategy that caps $d_{\max}$ at 4 while guaranteeing a height h that is logarithmic in n, the total number of entries stored in the map.

11.5.2 (2,4)-Tree Operations

One form of a multiway search tree that keeps the tree balanced while using small secondary data structures at each node is the $(\mathbf{2},\mathbf{4})$ ***tree***, also known as a 2-4 tree or 2-3-4 tree. This data structure achieves these goals by maintaining two simple properties (see Figure 11.23):

***Size Property*:** Every internal node has at most four children.

***Depth Property*:** All the external nodes have the same depth.

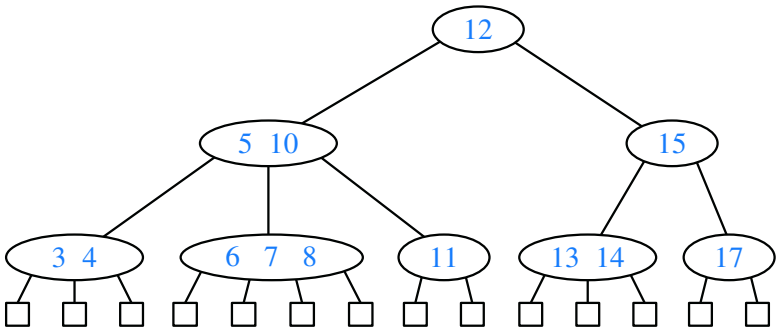

Figure 11.23: A $(2,4)$ tree.

Again, we assume that external nodes are empty and, for the sake of simplicity, we describe our search and update methods assuming that external nodes are real nodes, although this latter requirement is not strictly needed.

Enforcing the size property for $(2,4)$ trees keeps the nodes in the multiway search tree simple. It also gives rise to the alternative name "2-3-4 tree," since it implies that each internal node in the tree has 2, 3, or 4 children. Another implication of this rule is that we can represent the secondary map stored at each internal node using an unordered list or an ordered array, and still achieve $O(1)$-time performance for all operations (since $d_{\max} = 4$). The depth property, on the other hand, enforces an important bound on the height of a $(2,4)$ tree.

Proposition 11.7: *The height of a $(2,4)$ tree storing n entries is $O(\log n)$.*

Justification: Let h be the height of a $(2,4)$ tree T storing n entries. We justify the proposition by showing the claim

$$\frac{1}{2}\log(n+1) \leq h \leq \log(n+1). \tag{11.9}$$

To justify this claim note first that, by the size property, we can have at most 4 nodes at depth 1, at most 4^2 nodes at depth 2, and so on. Thus, the number of external nodes in T is at most 4^h. Likewise, by the depth property and the definition

of a $(2,4)$ tree, we must have at least 2 nodes at depth 1, at least 2^2 nodes at depth 2, and so on. Thus, the number of external nodes in T is at least 2^h. In addition, by Proposition 11.6, the number of external nodes in T is $n+1$. Therefore, we obtain

$$2^h \leq n+1 \leq 4^h.$$

Taking the logarithm in base 2 of the terms for the above inequalities, we get that

$$h \leq \log(n+1) \leq 2h,$$

which justifies our claim (Formula 11.9) when terms are rearranged. ■

Proposition 11.7 states that the size and depth properties are sufficient for keeping a multiway tree balanced. Moreover, this proposition implies that performing a search in a $(2,4)$ tree takes $O(\log n)$ time and that the specific realization of the secondary structures at the nodes is not a crucial design choice, since the maximum number of children $d_{\max}$ is a constant.

Maintaining the size and depth properties requires some effort after performing insertions and deletions in a $(2,4)$ tree, however. We discuss these operations next.

Insertion

To insert a new entry (k,v), with key k, into a $(2,4)$ tree T, we first perform a search for k. Assuming that T has no entry with key k, this search terminates unsuccessfully at an external node z. Let w be the parent of z. We insert the new entry into node w and add a new child y (an external node) to w on the left of z.

Our insertion method preserves the depth property, since we add a new external node at the same level as existing external nodes. Nevertheless, it may violate the size property. Indeed, if a node w was previously a 4-node, then it would become a 5-node after the insertion, which causes the tree T to no longer be a $(2,4)$ tree. This type of violation of the size property is called an ***overflow*** at node w, and it must be resolved in order to restore the properties of a $(2,4)$ tree. Let $c_1,\ldots,c_5$ be the children of w, and let $k_1,\ldots,k_4$ be the keys stored at w. To remedy the overflow at node w, we perform a ***split*** operation on w as follows (see Figure 11.24):

- Replace w with two nodes w' and w'', where
  - w' is a 3-node with children c_1,c_2,c_3 storing keys k_1 and k_2.
  - w'' is a 2-node with children c_4,c_5 storing key k_4.
- If w is the root of T, create a new root node u; else, let u be the parent of w.
- Insert key k_3 into u and make w' and w'' children of u, so that if w was child i of u, then w' and w'' become children i and $i+1$ of u, respectively.

As a consequence of a split operation on node w, a new overflow may occur at the parent u of w. If such an overflow occurs, it triggers in turn a split at node u. (See Figure 11.25.) A split operation either eliminates the overflow or propagates it into the parent of the current node. We show a sequence of insertions in a $(2,4)$ tree in Figure 11.26.

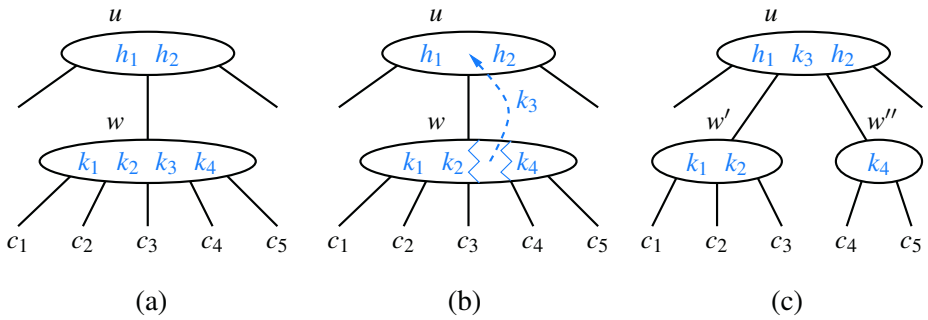

Figure 11.24: A node split: (a) overflow at a 5-node w; (b) the third key of w inserted into the parent u of w; (c) node w replaced with a 3-node w' and a 2-node w''.

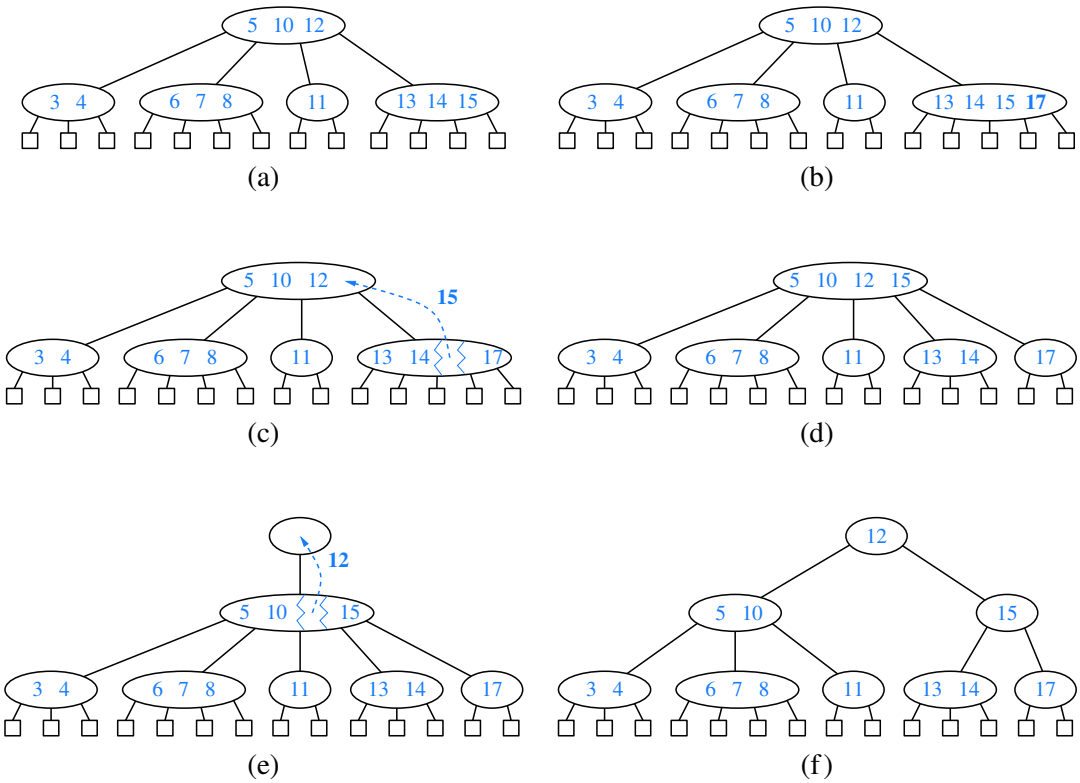

Figure 11.25: An insertion in a $(2,4)$ tree that causes a cascading split: (a) before the insertion; (b) insertion of 17, causing an overflow; (c) a split; (d) after the split a new overflow occurs; (e) another split, creating a new root node; (f) final tree.

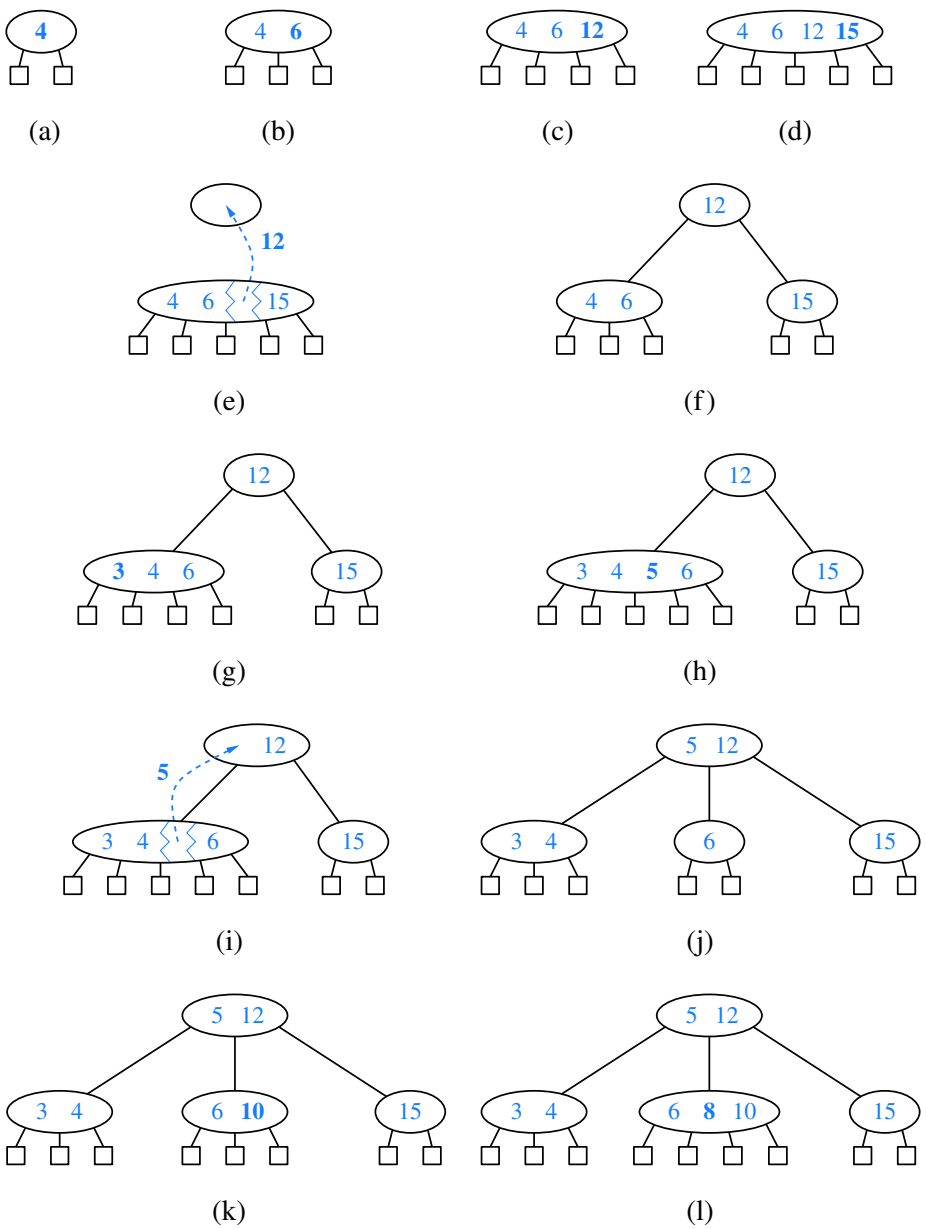

Figure 11.26: A sequence of insertions into a $(2,4)$ tree: (a) initial tree with one entry; (b) insertion of 6; (c) insertion of 12; (d) insertion of 15, which causes an overflow; (e) split, which causes the creation of a new root node; (f) after the split; (g) insertion of 3; (h) insertion of 5, which causes an overflow; (i) split; (j) after the split; (k) insertion of 10; (l) insertion of 8.

Analysis of Insertion in a (2,4) Tree

Because $d_{\max}$ is at most 4, the original search for the placement of new key k uses $O(1)$ time at each level, and thus $O(\log n)$ time overall, since the height of the tree is $O(\log n)$ by Proposition 11.7.

The modifications to a single node to insert a new key and child can be implemented to run in $O(1)$ time, as can a single split operation. The number of cascading split operations is bounded by the height of the tree, and so that phase of the insertion process also runs in $O(\log n)$ time. Therefore, the total time to perform an insertion in a $(2,4)$ tree is $O(\log n)$.

Deletion

Let us now consider the removal of an entry with key k from a $(2,4)$ tree T. We begin such an operation by performing a search in T for an entry with key k. Removing an entry from a $(2,4)$ tree can always be reduced to the case where the entry to be removed is stored at a node w whose children are external nodes. Suppose, for instance, that the entry with key k that we wish to remove is stored in the i^{th} entry (k_i, v_i) at a node z that has internal children. In this case, we swap the entry (k_i, v_i) with an appropriate entry that is stored at a node w with external children as follows (see Figure 11.27d):

1. We find the rightmost internal node w in the subtree rooted at the i^{th} child of z, noting that the children of node w are all external nodes.
2. We swap the entry (k_i, v_i) at z with the last entry of w.

Once we ensure that the entry to remove is stored at a node w with only external children (because either it was already at w or we swapped it into w), we simply remove the entry from w and remove the external node that is the i^{th} child of w.

Removing an entry (and a child) from a node w as described above preserves the depth property, for we always remove an external child from a node w with only external children. However, in removing such an external node, we may violate the size property at w. Indeed, if w was previously a 2-node, then it becomes a 1-node with no entries after the removal (Figure 11.27a and d), which is not allowed in a $(2,4)$ tree. This type of violation of the size property is called an ***underflow*** at node w. To remedy an underflow, we check whether an immediate sibling of w is a 3-node or a 4-node. If we find such a sibling s, then we perform a ***transfer*** operation, in which we move a child of s to w, a key of s to the parent u of w and s, and a key of u to w. (See Figure 11.27b and c.) If w has only one sibling, or if both immediate siblings of w are 2-nodes, then we perform a ***fusion*** operation, in which we merge w with a sibling, creating a new node w', and move a key from the parent u of w to w'. (See Figure 11.27e and f.)

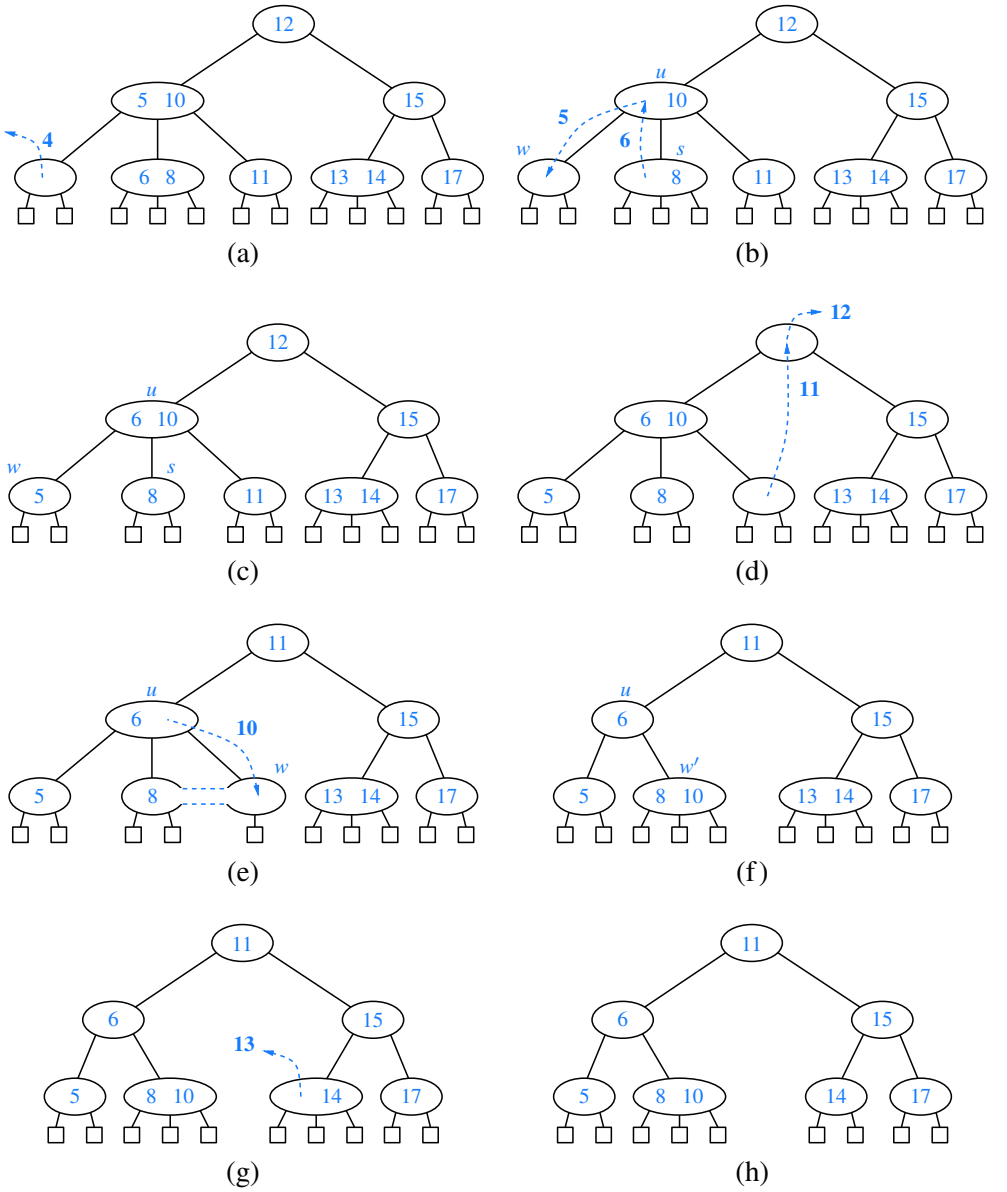

Figure 11.27: A sequence of removals from a $(2,4)$ tree: (a) removal of 4, causing an underflow; (b) a transfer operation; (c) after the transfer operation; (d) removal of 12, causing an underflow; (e) a fusion operation; (f) after the fusion operation; (g) removal of 13; (h) after removing 13.

A fusion operation at node w may cause a new underflow to occur at the parent u of w, which in turn triggers a transfer or fusion at u. (See Figure 11.28.) Hence, the number of fusion operations is bounded by the height of the tree, which is $O(\log n)$ by Proposition 11.7. If an underflow propagates all the way up to the root, then the root is simply deleted. (See Figure 11.28c and d.)

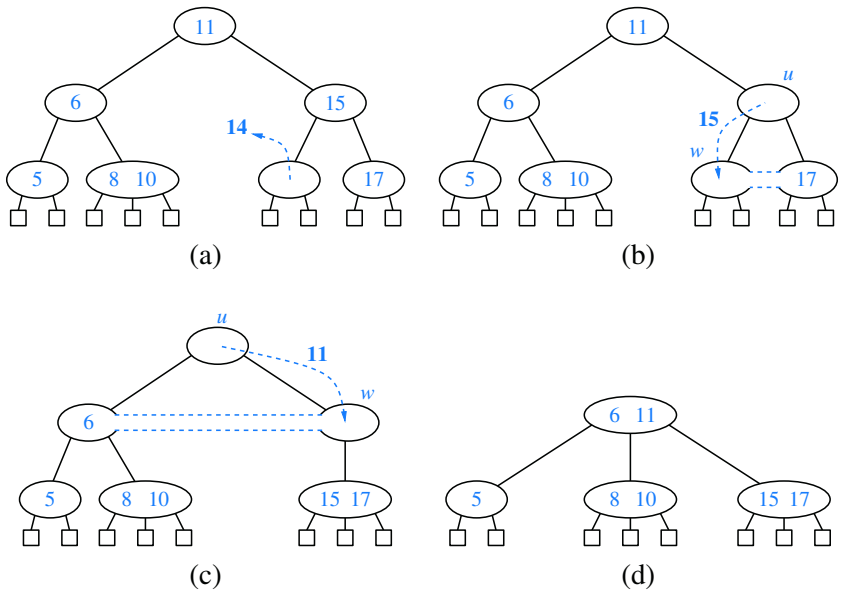

Figure 11.28: A propagating sequence of fusions in a $(2,4)$ tree: (a) removal of 14, which causes an underflow; (b) fusion, which causes another underflow; (c) second fusion operation, which causes the root to be removed; (d) final tree.

Performance of (2,4) Trees

The asymptotic performance of a $(2,4)$ tree is identical to that of an AVL tree (see Table 11.2) in terms of the sorted map ADT, with guaranteed logarithmic bounds for most operations. The time complexity analysis for a $(2,4)$ tree having n key-value pairs is based on the following:

- The height of a $(2,4)$ tree storing n entries is $O(\log n)$, by Proposition 11.7.
- A split, transfer, or fusion operation takes $O(1)$ time.
- A search, insertion, or removal of an entry visits $O(\log n)$ nodes.

Thus, $(2,4)$ trees provide for fast map search and update operations. $(2,4)$ trees also have an interesting relationship to the data structure we discuss next.